

Assignment: Collaborative Food Ordering App

Goal

Build a mobile application that allows users to place individual orders or participate in a real-time Group Order session. The focus is on how you handle state across multiple devices and manage live data synchronization.

Frontend: Flutter

Backend: Node.js/Golang

App Flow & Features

Order Modes

- Normal Order: Standard solo shopping experience.
- Group Order: A shared session where multiple people add items to a single cart simultaneously.

The Group Session

- Host: Can start a "Group Order" which generates a unique Join Code.
- Participants: Join via the code and enter a Display Name to identify themselves in the group.

Real-Time Collaborative Cart

- Live Sync: Implement real-time synchronization so that when any user adds an item or changes a quantity, the change is reflected instantly for everyone else in the group.
- Item Attribution: The cart must clearly indicate which user added each item (the specific UI implementation is up to you).
- Inventory & Stock: Handle item availability in real-time. For example, if User 1 adds the last available units of an item to their cart, it must immediately appear as "Out of Stock" for User 2. If User 1 removes them, the stock must be released and become available for others.

- Conflict Handling: If two users are viewing the same item and one updates the quantity, the other user's UI should update the count smoothly without interrupting their current scroll position or interaction.
- User Status: A list of active participants in the session showing their "Ready" status.

Checkout Logic

- Each participant must toggle a "Ready" button once they are done.
- The UI must clearly show who is "Ready" and who is "Still Browsing."
- The "Place Order" button should only become active for the Host once every participant has marked themselves as "Ready."

Technical Requirements

- Stack: Flutter + Golang/NodeJS.
- Database: Your choice (PostgreSQL, MongoDB, Redis, etc.).
- Communication: Real-time sync (WebSockets or better).
- State Management: **(ANY)**. Ensure you are handling stream-based data efficiently to avoid unnecessary UI rebuilds.

Deliverables

- GitHub Repos: Both Frontend and Backend code.
- APK: A debug or release build for testing.
- README: Please include:
 - a. Setup and run instructions (mention specific versions used).
 - b. A brief explanation video of how you architected the sync logic.
 - c. A list of packages used and why you chose them.
 - d. Any assumptions you made during development.

Timeline

4 Days

Submission

Email: ishtee@inminutes.now or deepraj@inminutes.now

Subject: [FullStack Developer Intern Assignment]: Your Name

Include your repo links and the APK.

Note

This assignment is designed to see how you handle real-time concurrency and distributed state. While the technical logic is the priority, we are looking for a polished and professional UI that feels like a real product. The experience should be smooth, ensuring data stays in sync seamlessly across devices.